

HrFlow.ai

18/03/2025

Victor Guichard
ENS Paris-Saclay

guichardvictor@gmail.com

Eliott Vigier
ENS Paris-Saclay

eliott.vigier@gmail.com

1. Introduction

Recommending suitable jobs to candidates without using personal information like resumes is a challenging task. Instead of relying on profile data, HrFlow.ai's goal is to predict job preferences based on a candidate's browsing history, which reflects the sequence of jobs they have viewed or applied to.

Therefor, the objective is to develop a model that predicts the top 10 jobs a candidate is likely to engage with next and whether their next action will be to view or apply.

To address this problem, we will explore different approaches, including collaborative filtering (CF), online machine learning, and graph-based methods.

2. Data

The dataset consists of approximately 18k candidate sessions, with around 16k used for training and 2k for testing. Each session contains the browsing history of job IDs viewed by the candidate, along with the action taken, which can be either "view" or "apply."

In total, there are around 22k unique job listings. Additionally, a `job_listing.json` file links each job ID to a title and a summary, providing information about the jobs.

We built from the job listing an embedding matrix using the multilingual e5 large model [1], in order to obtain a rich representation of the different jobs. These embeddings were used by multiple algorithms to provide contextual information about a job id.

3. Metrics

The model will be evaluated based on two key metrics:

- **Mean Reciprocal Rank (MRR):** Measures the average of the reciprocal ranks of the correct job within the top 10 predicted jobs. A higher MRR indicates that the correct job is ranked higher in the recommendations.
- **Accuracy:** Evaluates how well the model predicts whether the candidate's next action will be an *apply* or *view*, as indicated by the "action" variable.

The final score is computed as a weighted combination of these two metrics:

- 70% MRR
- 30% Accuracy on the "action" variable

We also used the **HitRate@k** metric which translates into the top k accuracy. This metric computes the number of times where the correct job id is among the top k predicted jobs (ranked by predicted scores). This helps identify how effective the algorithm is at ranking the true job. Similarly to the MRR, k is set at 10.

4. Baseline : Collaborative Filtering (CF)

CF is a technique used to make predictions based on the preferences and behaviors of similar users. Here it is used to recommend the most popular jobs but also to predict whether a candidate will apply for one of these jobs.

4.1. Steps involved

1. **User-Job interaction matrix:** We represent user-job interactions with a matrix $\mathbf{R}$, where $R_{ui} = 1$ if user u viewed or applied to job i , and $R_{ui} = 0$ otherwise.
2. **Finding similar users:** Similarity between users is computed using *cosine similarity*:

$$S_{uv} = \frac{\sum_i R_{ui} \cdot R_{vi}}{\sqrt{\sum_i R_{ui}^2} \cdot \sqrt{\sum_i R_{vi}^2}}$$

3. **Predicting job popularity:** For a candidate c , the predicted score for job j is the average interaction score among the top k similar users:

$$P_{cj} = \frac{\sum_{u \in U_c} R_{uj}}{k}$$

4. **Predicting application intent:** The predicted intent score is computed as the average of the top 10 job scores:

$$P_{c, \text{average}} = \frac{1}{10} \sum_{j=1}^{10} P_{cj}$$

The candidate is predicted to apply if the score exceeds a threshold θ :

$$\text{applies_for} = \begin{cases} 1, & \text{if } P_{c,\text{average}} \geq \theta \\ 0, & \text{if } P_{c,\text{average}} < \theta \end{cases}$$

4.2. Hyperparameter tuning

We experimented with different values of k (number of similar users) and θ (application threshold). The best results were observed for θ between 0.3 and 0.4, beyond which the scores started to decline. The performance on the training set for $k = 10$ is summarized in Table 1:

k	Threshold (θ)	MRR	Accuracy	Final Score
10	0.2	0.0204	0.5000	0.1643
10	0.3	0.0204	0.6078	0.1966
10	0.4	0.0204	0.6169	0.1993
10	0.5	0.0204	0.6069	0.1964

Table 1. Performance of collaborative filtering on training set

The best overall results were obtained with $k = 21$, which gave the following scores:

- **MRR** = 0.0218
- **Accuracy** = 0.6249
- **Final score** = 0.2027

Beyond $k = 21$, both MRR and accuracy started to decline.

We also experimented with changing the definition of R_{ui} to assign different values for "view" and "apply" events instead of using $R_{ui} = 1$ for both. However, this approach led to a decrease in performance.

4.3. Evaluation on test set

We tested this configuration on the test set. However, the final score was around 0.158, which is below the baseline score of 0.192.

To improve the results, we merged the training and test sets, as there were no restrictions against doing so. Using the same hyperparameters, we obtained a final score of 0.194, which exceeds the baseline.

5. Combining Singular Value Decomposition (SVD) with CF

5.1. Modifications to the baseline pipeline

Key modifications in the SVD-based approach are:

1. **Matrix factorization via SVD:** We decompose the interaction matrix $\mathbf{R}$ using SVD:

$$\mathbf{R} \approx \mathbf{U} \Sigma \mathbf{V}^T$$

and approximate it by keeping only the top l singular values:

$$\mathbf{R}_{\text{svd}} \approx \mathbf{U}_1 \Sigma_1 \mathbf{V}_1^T$$

2. **Prediction based on $\mathbf{R}_{\text{svd}}$:** For each candidate session, we rank jobs by the reconstructed scores and select the top n jobs. The predicted application intent is determined by comparing the average score of these top n jobs with a threshold θ :

$$\text{applies_for} = \begin{cases} 1, & \text{if } \frac{1}{n} \sum_{j \in \text{Top}_n} \hat{R}_{cj} \geq \theta, \\ 0, & \text{otherwise.} \end{cases}$$

5.2. Experimental results

We made some experiments to evaluate the impact of assigning different weights to "apply" and "view" events. Unlike the baseline CF model, where treating "apply" and "view" events equally gave the best results, changing these weights had an impact on performance. The results on the training set are summarized in Table 2:

Dims (l)	Thres.	Apply W.	View W.	Score
20	0.3	1.0	1.0	0.2002
20	0.3	2.0	1.0	0.2118
20	0.3	1.0	2.0	0.1898

Table 2. Performance with different weights for "Apply" and "View"

Increasing the weight for "apply" events to 2.0 improved the final score from 0.2002 to 0.2118, while increasing the weight for "view" events to 2.0 led to a decrease in performance.

Moreover, since the number of neighbors k is no longer used in the SVD-based approach, the most important parameters are the number of latent dimensions and the threshold θ . By increasing the number of latent dimensions to 150, setting θ to 0.6, and assigning a higher weight of 100 to "apply" events, the model achieved the best performance:

- **MRR** = 0.0238
- **Accuracy** = 0.7389
- **Final score** = 0.2383

6. Graph

The Graph-Based Job recommendation algorithm is designed to provide personalized job recommendations by leveraging both session transition data and job feature similarities. This hybrid approach aims to enhance the relevance and accuracy of job recommendations by considering the sequence of user interactions and the inherent features of the jobs. This Graph-Based approach is very similar to collaborative filtering but allows for a fast recommendation system as well as a more in detailed session representation.

6.1. Graph Representation

The algorithm utilizes a directed graph to capture transitions between jobs within user sessions. Nodes in the graph

represent jobs, while edges represent transitions from one job to another. Edges are weighted based on the type of action (e.g., 'apply' or 'view') and normalized to represent transition probabilities.

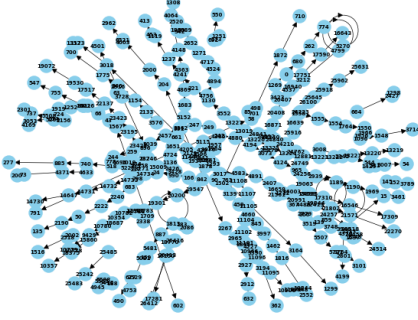


Figure 1. Graph representation of the session and the different transitions between jobs from a random sample of 50 sessions

The ratio of 'apply' actions to total actions is tracked for each job to predict the likelihood of a job being applied to in future recommendations.

When the graph is constructed by going through all the sessions, edge weights are normalized to represent transition probabilities, ensuring that the sum of outgoing edge weights for each node equals 1.

6.2. Graph-Based Scoring

Given a sequence of past job interactions represented as $(job_id, action)$ pairs, the algorithm iterates through each job in the sequence. If a job node exists in the directed graph G , its successors (neighboring jobs) are examined to determine their relevance.

For each neighbor job n of a visited job j , a transition weight is computed based on two factors:

- **Graph Transition Weight:** If an edge exists between j and n labeled with the action type (e.g., clicked, applied), its weight is retrieved and contributes to the score. This is scaled by a factor α and weighted by the interaction's position e in the sequence:

$$S(n)+ = \alpha \cdot w(j, n) \cdot \frac{e + 1}{|job_ids|} \quad (1)$$

where $w(j, n)$ is the edge weight in the graph.

- **Embedding-Based Similarity:** The similarity between job embeddings is computed as the dot product between the embedding vectors:

$$sim(j, n) = \mathbf{v}_j \cdot \mathbf{v}_n^T \quad (2)$$

If this similarity exceeds a predefined threshold, it con-

tributes to the score, weighted by $(1 - \alpha)$:

$$S(n)+ = (1 - \alpha) \cdot sim(j, n) \cdot \frac{e + 1}{|job_ids|} \quad (3)$$

6.3. Ranking and Recommendation

Once scores are computed for all candidate jobs, they are sorted in descending order based on their score values. Jobs that have already been interacted with are excluded from the final ranked list:

$$ranked_{jobs} = sort(n, S(n)) \mid n \notin job_ids \quad (4)$$

The final ranked list of jobs is presented as recommendations to the user, prioritizing jobs with the highest relevance scores.

6.4. Results

We report the quality of the algorithm in table 3. And observed an accuracy in the action detection of 72.36%.

α	hitrate	mrr
0	17.4	0.0770
0.2	17.5	0.0778
1	12.6	0.0521

Table 3. Results of the graph base algorithm. α at 0 translates as the transition weights are not used whereas α at 1 the similarities are not used.

As we can see from our results, the transition weights are not improving our accuracies. One possible explanation could be that the transition weights are not properly tuned, or that taking the most probable successor is not enough to capture the user intent.

One advantage of such method is efficiency. On the training data, the graph is built in a few milliseconds. Once the graph is built, recommending jobs for a session takes 3ms on a laptop cpu.

7. Improved collaborative filtering with job embeddings

7.1. Job recommendation

Following the SVD enriched collaborative filtering,

To enhance job representations, we integrate job embeddings with the SVD-derived job factors. Given the embedding matrix $E \in \mathbb{R}^{n \times d}$, we construct an augmented job representation:

$$J = [V_k, E] \quad (5)$$

where $J \in \mathbb{R}^{n \times (k+d)}$ concatenates the collaborative and content-based embeddings.

Finally, for each session i , we generate job recommendations by computing the similarity between user and job

representations. To prevent recommending previously seen jobs, we mask out previously interacted items.

7.2. Next action prediction

To predict whether a user will apply to a recommended job, we train a logistic regression model using session-level features extracted from historical interactions. Given a feature vector $x \in \mathbb{R}^3$ representing total interactions, number of applications, and number of views, the probability of an apply action is modeled as:

$$P(y = 1|x) = \frac{1}{1 + e^{-\theta^T x}} \quad (6)$$

where θ represents the learned parameters of the logistic regression model and 1 corresponds to applying.

7.3. Results

This approach obtains a 0.042 MRR and 10% hit rate. Our action accuracy reaches 77.59%. Integrating job embedding improves both the MRR and the hit rate compared to the collaborative filtering of the baseline. However, the graph-based algorithms still outperform it.

8. Conclusion

Model	MRR	Hit Rate	Accuracy
Baseline CF	0.0218	NA	62.49%
SVD + CF	0.0238	NA	73.89%
Graph-based	0.0778	17.5%	72.36%
Job Embedding + CF	0.0420	10.0%	77.59%

Table 4. Summary of performance across different models on the train dataset

The Collaborative Filtering (CF) model achieved a final score of 0.194 on the test set. By integrating SVD and job embeddings into the CF model, we increased the final score to 0.250. Finally, the best performance was obtained with the Graph-based model, which achieved a final score of 0.289.

Overall, the results show that incorporating additional structure through embeddings and graph-based methods improves both the relevance and accuracy of recommendations. The graph-based model stands out as the most effective approach.

Future works could include improving our graph-based approach with better defined transition weights, different scaling based on the position in the sequence, or even a graph neural network to predict both the most likely next job and action. Moreover, simply mixing the action prediction strategy done in the CF model with our graph-based model could obtain even better results than the one submitted.

References

- [1] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Multilingual e5 text embeddings: A technical report. *arXiv preprint arXiv:2402.05672*, 2024. 1